

ADO.NET – Short Notes

What is ADO.NET?

ADO.NET (ActiveX Data Objects .NET) is a data access technology provided by Microsoft that allows .NET applications to connect to databases and perform CRUD (Create, Read, Update, Delete) operations.

ADO.NET acts as a bridge between a .NET application and a database such as SQL Server, Oracle, MySQL, or PostgreSQL.

Why Use ADO.NET?

- Connect to databases
- Execute SQL queries
- Retrieve data
- Insert, Update, Delete records
- Work with Stored Procedures
- High performance and direct database access

ADO.NET Architecture

Application

|

v

SqlConnection

|

v

SqlCommand

|

+---- ExecuteReader()

|

+---- ExecuteNonQuery()

|

+---- ExecuteScalar()

|

v

Database

Main Components of ADO.NET

1. SqlConnection

Used to establish a connection with the database.

Responsibilities:

- Open database connection
- Close database connection

Example:

```
SqlConnection con = new SqlConnection(connectionString);
```

2. SqlCommand

Used to execute SQL statements.

Responsibilities:

- Execute SELECT statements
- Execute INSERT statements
- Execute UPDATE statements
- Execute DELETE statements
- Execute Stored Procedures

Example:

```
SqlCommand cmd = new SqlCommand(query, con);
```

3. SqlDataReader

Used to read records one row at a time.

Features:

- Fast
- Forward-only
- Read-only

Example:

```
SqlDataReader reader = cmd.ExecuteReader();
```

4. ExecuteNonQuery()

Used for:

- INSERT
- UPDATE
- DELETE

Example:

```
int rows = cmd.ExecuteNonQuery();
```

Returns the number of affected rows.

5. ExecuteReader()

Used for SELECT queries.

Example:

```
SqlDataReader reader = cmd.ExecuteReader();
```

Returns records from the database.

6. ExecuteScalar()

Used when a query returns a single value.

Example:

```
SELECT COUNT(*) FROM Books
```

```
int count = (int)cmd.ExecuteScalar();
```

Returns a single value.

7. Parameters

Used to pass values safely to SQL queries.

Example:

```
cmd.Parameters.AddWithValue("@Id", 1);
```

Benefits:

- Prevent SQL Injection
- Improve security
- Improve readability

CRUD Operations in ADO.NET

Create

```
INSERT INTO Books(...)
```

Read

```
SELECT * FROM Books
```

Update

```
UPDATE Books SET ...
```

Delete

```
DELETE FROM Books WHERE Id = 1
```

Advantages of ADO.NET

- Fast performance
- Direct database access
- Full control over SQL queries
- Supports multiple databases
- Suitable for enterprise applications

Disadvantages of ADO.NET

- More code compared to EF Core
- Manual SQL writing required
- Harder to maintain large applications
- Developer must manage connections and queries

ADO.NET vs EF Core

ADO.NET

- Write SQL manually
- Faster
- More code
- Full SQL control
- Suitable for legacy applications

EF Core

- Uses LINQ
- Slightly slower
- Less code
- SQL generated automatically
- Suitable for modern applications

Interview Questions

Q1. What is ADO.NET?

ADO.NET is a .NET technology used to connect applications with databases and perform data operations.

Q2. What is SqlConnection?

A class used to establish a connection with a database.

Q3. What is SqlCommand?

A class used to execute SQL queries and stored procedures.

Q4. What is SqlDataReader?

A forward-only, read-only object used to read records from a database.

Q5. Difference between ExecuteReader() and ExecuteNonQuery()?

ExecuteReader()

- Used for SELECT
- Returns SqlDataReader

ExecuteNonQuery()

- Used for INSERT, UPDATE, DELETE
- Returns affected row count

Q6. What is ExecuteScalar()?

Used when a query returns a single value such as COUNT, MAX, MIN, SUM, etc.

Q7. Why use Parameters?

To prevent SQL Injection attacks and securely pass values to SQL queries.

Summary

ADO.NET is the foundational data access technology in .NET.

It provides direct communication with databases using classes such as SqlConnection, SqlCommand, and SqlDataReader.

It offers high performance, full control over SQL queries, and is widely used in enterprise and legacy applications.

Entity Framework Core is built on top of concepts provided by ADO.NET.